

Application & Infrastructure Evolution

1. Big Picture — How Technology Evolved

Think about a **restaurant** 🍽️.

We can understand the evolution like this:

Physical Server
↓
Virtual Machine
↓
Containers
↓
Microservices

And at the application architecture level:

Monolithic Application
↓
Microservices Architecture

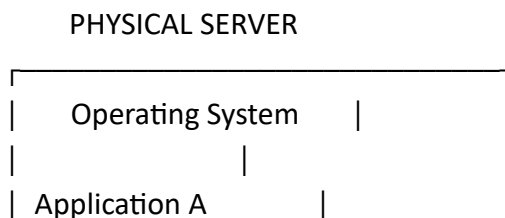
These are related, but **not exactly the same thing**:

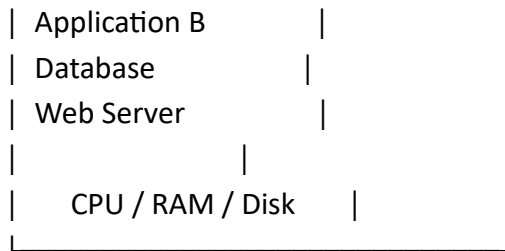
- **Physical server → VM → Container** = evolution of **infrastructure/application isolation**
- **Monolithic → Microservices** = evolution of **application architecture**

🟢 2. Physical Server — "One Big House"

Imagine you own a large house 🏠.

Everything is inside the same house:





Layman Example

Imagine a restaurant where:

- Kitchen
- Billing
- Storage
- Reception
- Dining area

are all running in **one building**.

If the building has a major problem, everything is affected.

Advantages

- ✓ Simple architecture
- ✓ Direct access to hardware
- ✓ Very good performance
- ✓ No virtualization overhead
- ✓ Easy to understand initially

Disadvantages

- ✗ Expensive hardware
- ✗ Resources can be underutilized
- ✗ Scaling is difficult
- ✗ One server failure can affect many applications
- ✗ Provisioning a new server takes time

✗ Applications can conflict with each other

DevOps Problem

Suppose:

Application A needs Java 8

Application B needs Java 17

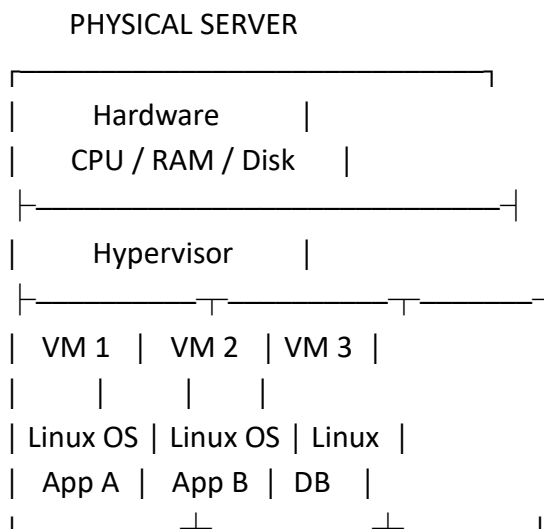
Both applications are installed on the same OS.

You can potentially have dependency/version conflicts.

3. Virtual Server — "Multiple Houses Inside One Building"

Virtualization changed the model.

Instead of buying one physical server for every application, we can divide one physical server into multiple **Virtual Machines (VMs)**.



Layman Example

Imagine one large apartment building 🏠.

Instead of one family using the entire building:

Building

- └─ Apartment 1 → Family A
- └─ Apartment 2 → Family B
- └─ Apartment 3 → Family C

Each family has its own apartment.

Similarly:

Physical Server

- └─ VM1 → Application A
- └─ VM2 → Application B
- └─ VM3 → Database

What is a Hypervisor?

A **hypervisor** is software that creates and manages virtual machines.

Examples:

- VMware ESXi
- Microsoft Hyper-V
- KVM

4. Advantages of Virtual Machines

1. Better resource utilization

One physical server can run multiple VMs.

2. Isolation

If one VM crashes:

VM1 ❌

VM2 ✅

VM3 ✅

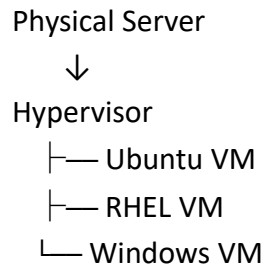
The other VMs can continue running.

3. Easy provisioning

You can create VMs from templates/images.

4. Different operating systems

For example:



5. Snapshots

VMs can often be snapshot and restored.

● 5. Disadvantages of Virtual Machines

The biggest issue:

Every VM requires its own Operating System.

VM1 → OS

VM2 → OS

VM3 → OS

VM4 → OS

This consumes:

- RAM
- CPU
- Disk
- Boot time

For example:

Physical Server



Hypervisor



VM1 → 4 GB RAM

VM2 → 4 GB RAM

VM3 → 4 GB RAM

VM4 → 4 GB RAM

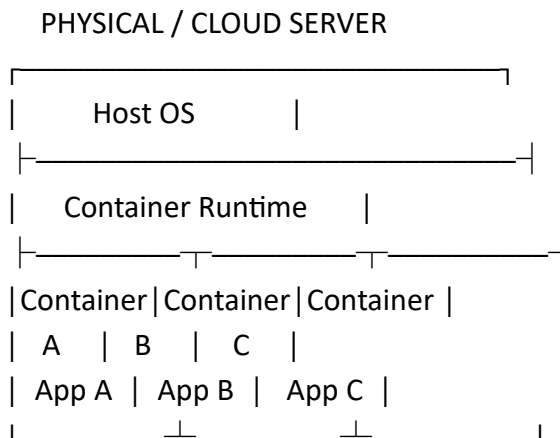
A lot of resources are spent just maintaining operating systems.



6. Containers — "Lightweight Rooms"

Containers solve some of these problems.

Instead of giving every application a complete OS, containers share the **host OS kernel** while keeping application processes isolated.



Popular container technology:

Docker



7. Container — Layman Example

Imagine an apartment again.

VM approach

Each apartment has almost everything needed to operate independently.

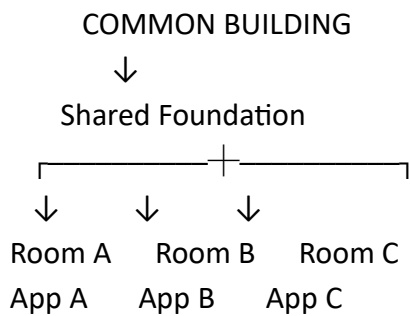
Apartment 1 → Full setup

Apartment 2 → Full setup

Apartment 3 → Full setup

Container approach

The building provides common infrastructure:



Each application gets an isolated space but shares common underlying infrastructure.

⚡ 8. Why Are Containers Lightweight?

Compare:

VM:

Application

Libraries

Guest OS

Virtual Hardware

Hypervisor

Physical Hardware

versus:

Container:

Application

Libraries

Container

Container Runtime

Host OS

Physical Hardware

The container doesn't need a separate full guest OS for every application.

Therefore containers generally:

- Start faster
- Use fewer resources
- Are easier to package
- Are easier to scale

9. Advantages of Containers

1. Lightweight

Containers generally require fewer resources than full VMs.

2. Fast startup

VM:

Boot OS



Start services



Start application

Container:

Start container



Application starts

3. Portability

The famous DevOps problem:

"It works on my machine!" 😊

Containers help package the application with its required dependencies.

Developer Laptop



Docker Image



Test



Production

4. Easy scaling

Suppose traffic increases:

Before:

Container 1

After:

Container 1

Container 2

Container 3

Container 4

5. Excellent for CI/CD

A pipeline can:

Code



Build



Create Docker Image



Test



Push Image



Deploy

10. Disadvantages of Containers

Containers aren't perfect.

1. Shared kernel

Containers on the same host share the host kernel.

Therefore isolation is different from having completely separate VMs.

2. Security requires careful configuration

You must properly manage:

- Images
- Privileges
- Secrets
- Network access
- Container permissions

3. Persistent storage can be complicated

Containers are often treated as ephemeral.

Container



Deleted



Application data could be lost

Therefore persistent storage needs to be designed separately.

4. Networking becomes more complicated at scale

With hundreds or thousands of containers:

Container

Container

Container

Container

...

Container

you need proper:

- Networking
- Service discovery
- Load balancing
- Monitoring
- Logging

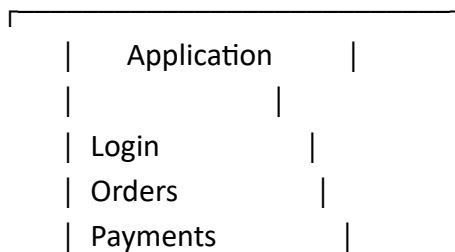
This leads us toward **Kubernetes** and other orchestration platforms.

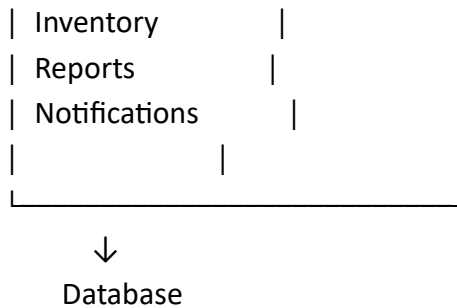
11. Monolithic Architecture

Now let's move from **infrastructure** to **application architecture**.

Imagine a restaurant where one person does everything:

MONOLITHIC APPLICATION

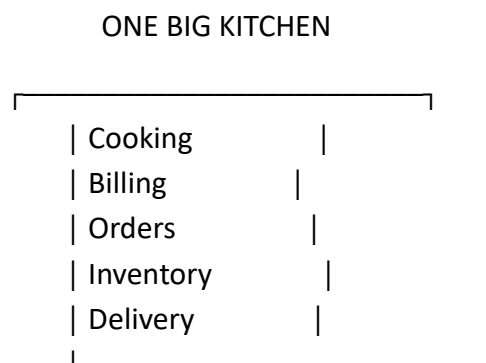




Everything is packaged as one application.

12. Monolithic Layman Example

Imagine a restaurant with one giant kitchen.



If the kitchen has a major problem, many restaurant operations are affected.

13. Advantages of Monolithic Architecture

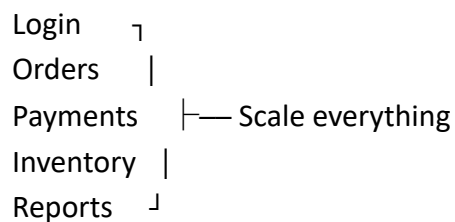
-  Simple to develop initially
-  Simple deployment
-  Easy local testing
-  Straightforward debugging
-  Easy communication between modules
-  Suitable for smaller applications

🔴 14. Disadvantages of Monolithic Architecture

1. Scaling problem

Suppose only the payment functionality needs more resources.

With a monolith, you may have to scale the **whole application**.



2. Large codebase

As the application grows, it becomes harder to understand and maintain.

3. Deployment risk

A small change in one module can require deployment of the whole application.

4. Technology limitations

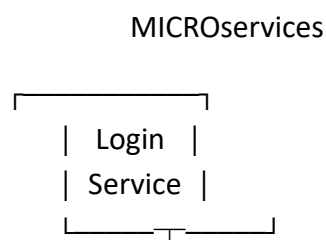
Different modules may have different technology requirements.

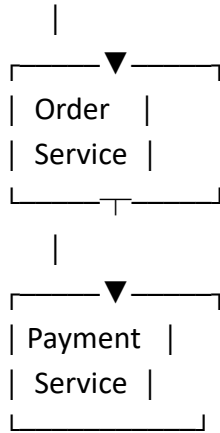
5. One major failure can affect the whole application

Depending on the design, failure in one component can impact the entire application.

🚀 15. Microservices Architecture

Now imagine the restaurant separates responsibilities.





Inventory Service

Notification Service

Each service focuses on a specific business capability.

16. Microservices — Layman Example

Think about a large restaurant chain.

Instead of one person doing everything:

Chef → Cooking

Cashier → Billing

Waiter → Orders

Storekeeper → Inventory

Delivery → Delivery

Each team has a focused responsibility.

Similarly:

Order Service



Handles orders

Payment Service



Handles payments

Inventory Service



Handles inventory

Notification Service



Handles notifications

Services communicate through APIs/messages.

● 17. Advantages of Microservices

Independent scaling

If payment receives huge traffic:

Order Service → 3 instances

Payment Service → 10 instances

Inventory Service → 3 instances

You don't necessarily need to scale everything equally.

Independent deployment

A change to one service can potentially be deployed independently.

Technology flexibility

Different services can potentially use different technologies when justified.

Order → Java

Payment → Python

Analytics → Go

Fault isolation

A failure in one service can potentially be isolated if the architecture has appropriate resilience mechanisms.

● 18. Disadvantages of Microservices

1. Complexity

Instead of:

1 Application

you may have:

50 Applications

Now you need to manage:

- Service discovery
- Networking
- Authentication
- Logging
- Monitoring
- Deployment
- Configuration
- Secrets

2. Distributed-system problems

Communication happens over networks.

Service A



Network

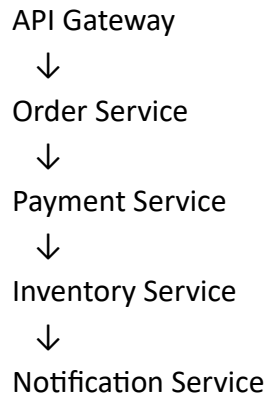


Service B

Network failures, latency, retries, and timeouts become important.

3. Debugging becomes harder

A single user request may travel through:



Tracing the failure requires good observability.

4. Operational overhead

Microservices often benefit from:

Docker
Kubernetes
CI/CD
Prometheus
Grafana
Centralized logging
Distributed tracing

19. Monolithic vs Microservices

Feature	Monolithic	Microservices
Application structure	One major application	Multiple services
Deployment	Usually together	Often independently
Scaling	Usually whole application	Service-level scaling
Complexity	Lower initially	Higher

Development	Simple initially	More coordination
Networking	Mostly internal	Network communication
Debugging	Easier initially	More complex
Deployment	Simple	More involved
Technology choice	Often shared	Can vary by service
Operations	Simpler	Requires stronger automation/observability
Best fit	Smaller/simple systems	Large systems with suitable boundaries

Important: Microservices are not automatically better. The right architecture depends on application size, team structure, operational maturity, scalability requirements, and business needs.

🔥 20. Physical vs VM vs Container

Feature	Physical Server	VM	Container
Hardware	Dedicated	Shared	Shared
OS	One OS	Guest OS per VM	Shares host kernel
Isolation	Physical	Strong	Process-level isolation
Startup	Slowest	Slower	Usually fastest
Resource usage	High	Higher	Lower
Portability	Low	Good	Very good
Density	Low	Medium	High
Scaling	Difficult	Easier	Very easy
Management	Hardware + OS	Hypervisor + OS	Runtime/orchestrator

Example

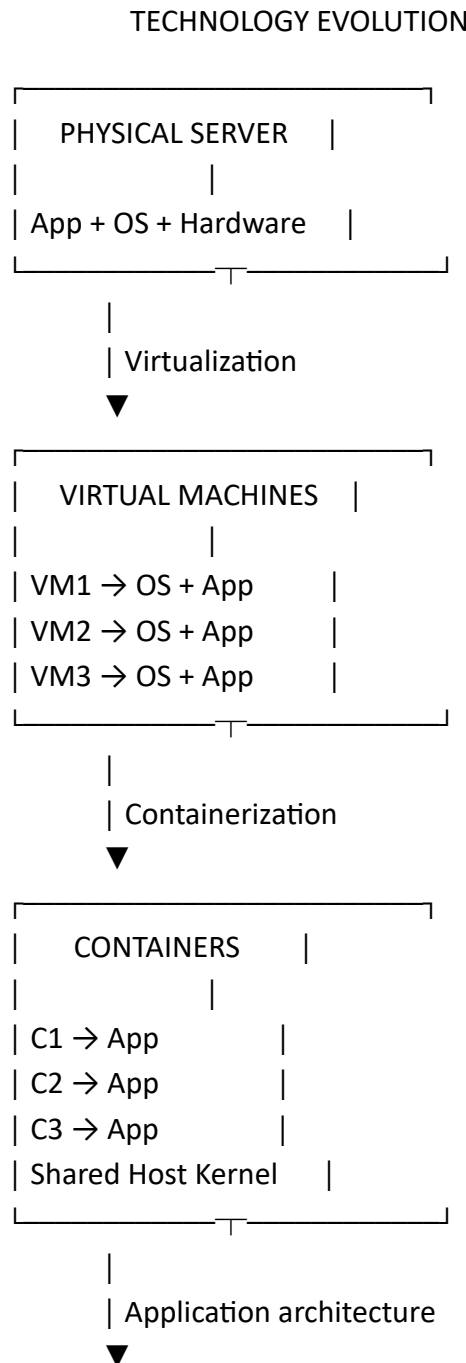
Bare metal

VMware/KVM

Docker

🧠 21. The Complete Evolution Story

This is the most important diagram to remember for interviews:



MICROSERVICES	
Order → Payment	
Inventory → User	
Notification → Search	

One-line interview answer

"We evolved from physical servers to virtual machines for better hardware utilization and isolation, then to containers for lightweight, portable and fast application deployment. At the application architecture level, monoliths evolved toward microservices in cases where independent scaling, deployment, and team ownership provided sufficient benefit to justify the additional distributed-system complexity."